

CVM++

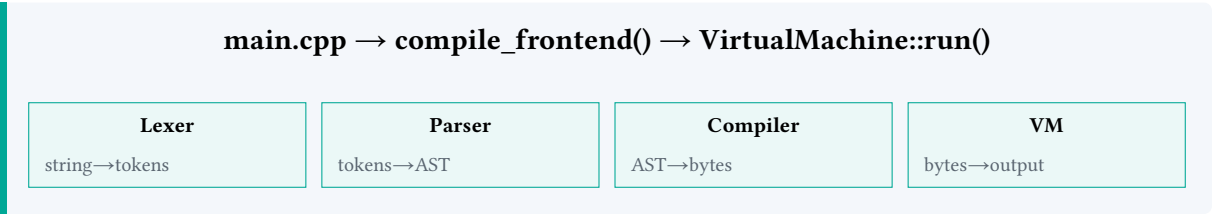
How cvmpp runs your program internally

5-page presentation · github.com/Dragon9-9/cvmpp

Satish Kumar Munda · github.com/Dragon9-9/cvmpp · C++17 Personal Project

Runtime overview

`./build/cvmpp` `examples/hello.cvm`



Stage	Input	Output
Lexer	Source text	vector of Token
Parser	Tokens	Program AST
Compiler	AST	BytecodeChunk
VM	Bytecode	Printed lines + errors

Front end (compile.cpp)

```
compile_frontend(source):  
  1. Lexer::tokenize()  
  2. Parser::parse()  
  3. Compiler::compile()  
  // main.cpp then: execute(chunk)
```

Parser output

Program
├ functions[] (fn defs)
└ statements[] (main script)

Precedence: eq → compare → add/sub → mul/
div → unary minus

Bytecode layout

Offset	Content
0	JUMP → main
...	function bodies
main	top-level code
end	HALT

Without entry JUMP → crash on LOAD_LOCAL.

Compiler control flow

Construct	Bytecode idea
if / else	JUMP_IF_FALSE, patch else; JUMP end
while	loop label, cond, body, jump back
let / =	eval expr; STORE_VAR or STORE_LOCAL
fn / call	metadata table; CALL index + argc
return	RETURN restores frame + value

Name pool

Globals: intern name → **u16** index (LOAD_VAR / STORE_VAR).

Locals: parameter slots → **u8** (LOAD_LOCAL / STORE_LOCAL).

Virtual machine

State	Role
ip_	Index into bytecode
stack_	Operand stack
globals_	File-mode variables
frames_	Call stack
session_	REPL name→value map

Failsafes

1. Stack max 65,536
2. Step limit 1,000,000
3. Divide by zero → exit 2
4. Uninitialized var load
5. Stack under/overflow
6. Bad jump / truncated bytecode

Types: only int64_t and bool

Opcode reference

Stack: binary ops pop **b** (top) then **a**.

Op	Hex	Effect
PUSH_INT	01	push i64
LOAD/STORE_VAR	10/11	global u16 idx
LOAD/STORE_LOCAL	12/13	local u8 slot
ADD...GE	20–2A	math / compare
INPUT/PRINT	30/31	stdin / output
JUMP/JIF_FALSE	40/41	u32 offset
CALL/RETURN	42/43	fn + argc / return
HALT	FF	stop

CALL / RETURN (4 steps)

1. Caller pushes arguments on stack.
2. CALL: pop args → frame.locals; save return_ip; ip = function address.
3. Function runs; nested CALL/RETURN as needed.
4. RETURN: pop value; restore ip; push result to caller.

Lexer and parser (detail)

Lexer emits

Keywords: let fn return if else while
print input true false

Operators: == != <= >= < > + - * /

Literals, identifiers, // comments, Eof

Errors: bad char, int overflow → phase LEXER

Parser builds

Program { functions[], statements[] }

Function body must be { block }

Call: name(expr, ...) — arity checked at compile

Errors: bad assign target, unbalanced {} →
phase PARSER

Execution traces

Trace A: print 1 + 2;

Step	Instruction	Stack
1	PUSH_INT 1	[1]
2	PUSH_INT 2	[1,2]
3	ADD	[3]
4	PRINT	output 3

Trace B: print factorial(5);

1. Main: push 5, CALL factorial
2. Recursive frames until n <= 1 returns 1
3. Unwind multiplies: 1→2→6→24→120
4. PRINT shows 120

AST nodes (summary)

Expressions	Examples
Literals	int, bool
Binary / Unary	+ - == < ...
Call / Input	f(a) input

Statements	Examples
let / assign	declare, update
if / while	control flow
print / return	I/O, fn exit
Block	{ ... }

REPL vs file · Debug

	File	REPL
Globals	Reset each run	Persist via VmSession
Storage	Indexed array	Map by name

Debug pipeline

```
./build/cvmp -d examples/hello.cvm
```

1. Token table
2. AST tree
3. Bytecode disasm
4. Runtime output

Error phases: LEXER · PARSER · COMPILER · VM · REPL

Colored message + source caret — process does not crash on user errors.

